

Лабораторна робота №6

з теми «Написання тестів для користувацьких
сценаріїв»

Виконала:

ст. гр. ІП-22-1

Чемна (Микитин) О. І.

Перевірив:

Храбатин Р. І.

Мета: Навчитися створювати тести для перевірки користувацьких сценаріїв, розібратися в існуючих інструментах для створення тестів для перевірки користувацьких сценаріїв та навчитися їх обирати.

Короткі теоретичні відомості:

E2E-тестування, або наскрізне тестування, використовується для перевірки роботи програмного продукту з точки зору кінцевого користувача.

На відміну від модульних тестів, які перевіряють окремі функції, та UI-тестів, які перевіряють окремі елементи інтерфейсу, E2E-тести перевіряють повний шлях користувача в системі. Такий підхід дозволяє переконатися, що різні частини вебсайту працюють разом коректно.

Користувацький сценарій складається з послідовності дій, які виконує користувач для досягнення певної мети. Наприклад, користувач може відкрити сайт, перейти в каталог, знайти потрібну книжку, відкрити сторінку товару або скористатися пошуком. Автоматизований E2E-тест імітує такі дії в браузері та перевіряє, чи отриманий результат відповідає очікуваному.

Для створення таких тестів можна використовувати інструменти автоматизації браузера. У цій роботі використовується Puppeteer разом із Jest.

Puppeteer дозволяє відкривати сторінки, натискати на елементи, отримувати текст сторінки, перевіряти URL та взаємодіяти з вебінтерфейсом, а Jest використовується для запуску тестів і перевірки очікуваних результатів.

Завдання:

Завдання перше:

Оберіть ресурс який буде протестовано. Можна використовувати власно написані ресурси, але прохання надати посилання на репозиторій, для попереднього налаштування тестового середовища, щоб перевірити роботу написаного тестового рішення.

Завдання друге:

Написати реалізацію трьох користувацьких сценаріїв до обраного ресурсу. Прохання, коротко перед реалізацією опишіть, що ви перевірятимете даним

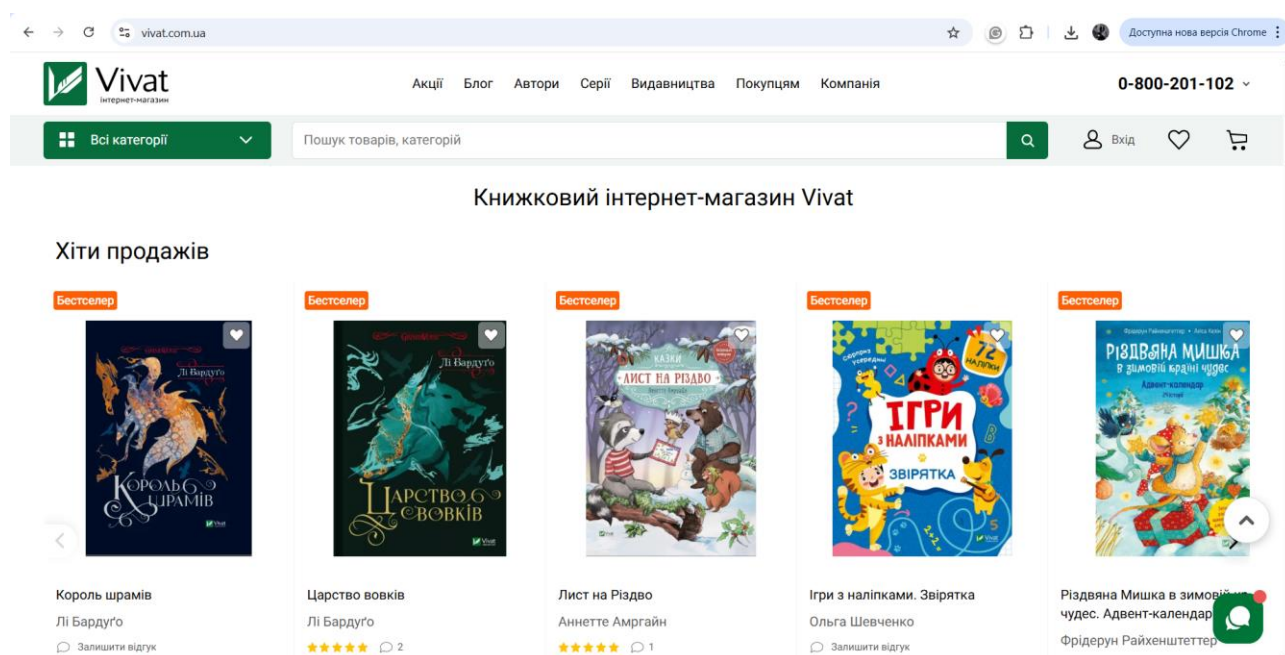
тестом. Це можна зробити у вигляді списку, як подано вище, оскільки в реалізації буде видно як саме ви проводите саме тестування.

Завдання третє:

Завантажити код на власний/навчальний репозиторій GitHub, або інший схожий ресурс та поділитися посиланням.

Виконання:

Для виконання роботи я обрала сайт Видавництва Віват:



Я обрала цей ресурс, оскільки він має зрозумілу структуру, головну сторінку, каталог книжок, сторінки товарів і навігаційні елементи. Це дозволяє створити користувацькі сценарії, які імітують реальні дії відвідувача інтернет-книгарні.

Опис користувацьких сценаріїв

№	Сценарій	Що перевіряється
1	Пошук книжки на сайті	Користувач відкриває сайт, знаходить поле пошуку, вводить запит і отримує сторінку з результатами
2	Перехід у категорію книжок та відкриття товару	Користувач переходить у каталог, знаходить посилання на товар і відкриває сторінку книжки

3	Перегляд товару та спроба додавання до кошика	Користувач відкриває сторінку книжки, перевіряє наявність інформації про товар і натискає кнопку додавання до кошика
---	---	--

Спочатку я створила окремий каталог для виконання лабораторної роботи:

```
oksanka@debian:~/Documents/automated_testing$ mkdir e2e-testing
oksanka@debian:~/Documents/automated_testing$ cd e2e-testing
```

Після цього я ініціалізувала Node.js-проект:

```
oksanka@debian:~/Documents/automated_testing/e2e-testing$ npm init -y
Wrote to /home/oksanka/Documents/automated_testing/e2e-testing/package.json:

{
  "name": "e2e-testing",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

oksanka@debian:~/Documents/automated_testing/e2e-testing$
```

Далі я встановила Jest для запуску тестів:

```
oksanka@debian:~/Documents/automated_testing/e2e-testing$ npm install --save-dev jest
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me

added 293 packages, and audited 294 packages in 19s

44 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
oksanka@debian:~/Documents/automated_testing/e2e-testing$
```

Після цього я встановила Puppeteer для автоматизації браузера:

```
oksanka@debian:~/Documents/automated_testing/e2e-testing$ npm install --save-dev puppeteer

added 66 packages, and audited 360 packages in 7s

48 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
oksanka@debian:~/Documents/automated_testing/e2e-testing$
```

Далі я відкрила файл package.json і змінила секцію scripts, щоб тести можна було запускати командою npm test:

```
6 | "scripts": {
7 |   "test": "jest"
8 | },
```

Після налаштування проєкту я створила файл для Е2Е-тестів. У файл `e2e.test.js` я додала тести для перевірки користувацьких сценаріїв на сайті Видавництва Віват:

```
package.json U e2e.test.js U X
e2e.test.js > ...
1  const puppeteer = require("puppeteer");
2
3  jest.setTimeout(90000);
4
5  const BASE_URL = "https://vivat.com.ua/";
6  const BOOKS_URL = "https://vivat.com.ua/category/knyhy/";
7
8  async function getBodyText(page) {
9    return page.$eval("body", (body) => body.innerText);
10 }
11
12 async function getLinksCount(page) {
13   return page.$eval("a", (links) => links.length);
14 }
15
16 async function clickFirstVisibleLinkWithText(page, textVariants) {
17   const clicked = await page.evaluate((variants) => {
18     const normalizedVariants = variants.map((item) => item.toLowerCase());
19
20     const links = Array.from(document.querySelectorAll("a"));
21
22     const target = links.find((link) => {
23       const text = link.innerText.toLowerCase().trim();
24       const href = link.href;
25
26       return (
27         href &&
28         text.length > 0 &&
29         normalizedVariants.some((variant) => text.includes(variant))
30       );
31     });
32
33     if (target) {
34       target.click();
35       return true;
36     }
37
38     return false;
39   }, textVariants);
40
41   return clicked;
42 }
43
44 async function clickFirstVisibleButtonWithText(page, textVariants) {
45   const clicked = await page.evaluate((variants) => {
46     const normalizedVariants = variants.map((item) => item.toLowerCase());
47
48     const elements = Array.from(
49       document.querySelectorAll("button, a, input[type='submit']"),
50     );
51
52     const target = elements.find((element) => {
53       const text = (
54         element.innerText ||
55         element.value ||
56         element.getAttribute("aria-label") ||
57         ""
58       )
59         .toLowerCase()
60         .trim();
```

```

61     return normalizedVariants.some((variant) => text.includes(variant));
62   });
63
64   if (target) {
65     target.click();
66     return true;
67   }
68
69   return false;
70 }, textVariants);
71
72 return clicked;
73 }
74
75
76 async function typeIntoSearch(page, searchText) {
77   const searchSelectors = [
78     "input[type='search']",
79     "input[name='q']",
80     "input[name='search']",
81     "input[placeholder*='Поиск']",
82     "input[placeholder*='поиск']",
83     "input[placeholder*='Search']",
84   ];
85
86   for (const selector of searchSelectors) {
87     const input = await page.$(selector);
88
89     if (input) {

```

```

89       if (input) {
90         await input.click({ clickCount: 3 });
91         await input.type(searchText);
92         await page.keyboard.press("Enter");
93         return true;
94       }
95     }
96
97     return false;
98   }
99
100   describe("Vivat E2E user scenarios", () => {
101     let browser;
102     let page;
103
104     beforeEach(async () => {
105       browser = await puppeteer.launch({
106         headless: true,
107         args: ["--no-sandbox", "--disable-setuid-sandbox"],
108       });
109
110       page = await browser.newPage();
111
112       await page.setViewport({
113         width: 1366,
114         height: 768,
115       });
116     });
117
118     afterEach(async () => {
119       await browser.close();
120     });

```

```

121
122     test("Scenario 1: user can search for books on the website", async () => {
123       await page.goto(BASE_URL, {
124         waitUntil: "networkidle2",
125       });
126
127       const searchWasFound = await typeIntoSearch(page, "книга");
128
129       expect(searchWasFound).toBe(true);
130     });

```

```

131     await page
132       .waitForNavigation({
133         waitUntil: "networkidle2",
134       })
135       .catch(() => null);
136
137     const bodyText = await getBodyText(page);
138     const linksCount = await getLinksCount(page);
139
140     expect(bodyText.toLowerCase()).toContain("книг");
141     expect(linksCount).toBeGreaterThan(5);
142   });
143
144   test("Scenario 2: user can open books category and navigate to a product page", async () => {
145     await page.goto(BOOKS_URL, {
146       waitUntil: "networkidle2",
147     });
148
149     const categoryText = await getBodyText(page);
150
151     expect(categoryText.toLowerCase()).toContain("книг");
152
153     const clickedProductLink = await clickFirstVisibleLinkWithText(page, [
154       "книга",
155       "роман",
156       "видання",
157       "комікс",
158       "манґа",
159     ]);
160

```

```

161     expect(clickedProductLink).toBe(true);
162
163     await page
164       .waitForNavigation({
165         waitUntil: "networkidle2",
166       })
167       .catch(() => null);
168
169     const productPageText = await getBodyText(page);
170
171     expect(productPageText.length).toBeGreaterThan(300);
172     expect(page.url()).toContain("vivat.com.ua");
173   });
174
175   test("Scenario 3: user can open product page and try to add a book to cart", async () => {
176     await page.goto(BOOKS_URL, {
177       waitUntil: "networkidle2",
178     });
179
180     const clickedProductLink = await clickFirstVisibleLinkWithText(page, [
181       "книга",
182       "роман",
183       "видання",
184       "комікс",
185       "манґа",
186     ]);
187
188     expect(clickedProductLink).toBe(true);
189

```

```

189
190     await page
191       .waitForNavigation({
192         waitUntil: "networkidle2",
193       })
194       .catch(() => null);
195
196     const productPageText = await getBodyText(page);
197
198     expect(productPageText.length).toBeGreaterThan(300);
199

```

```

199
200     const clickedBuyButton = await clickFirstVisibleButtonWithText(page, [
201         "купити",
202         "до кошика",
203         "в кошик",
204         "замовити",
205     ]);
206
207     expect(clickedBuyButton).toBe(true);
208
209     await new Promise((resolve) => setTimeout(resolve, 2000));
210
211     const updatedPageText = await getBodyText(page);
212
213     expect(updatedPageText.toLowerCase()).toMatch(
214         /кошик|купити|замовлення|товар/,
215     );
216 });
217 });
218

```

У цьому коді я використала допоміжні функції, щоб не дублювати однакові дії в різних сценаріях. Функція `getBodyText` отримує текст сторінки, `getLinksCount` рахує кількість посилань, `typeIntoSearch` шукає поле пошуку й вводить запит, а функції `clickFirstVisibleLinkWithText` та `clickFirstVisibleButtonWithText` допомагають натискати посилання або кнопки за текстом.

Перший сценарій перевіряв пошук книжки на сайті. Я відкрила головну сторінку, знайшла поле пошуку, ввела запит книга, натиснула Enter і перевірила, що після пошуку на сторінці є текст, пов'язаний із книжками, та достатня кількість посилань.

Другий сценарій перевіряв перехід у категорію книжок і відкриття сторінки товару. Я відкрила сторінку категорії книжок, перевірила наявність тексту, пов'язаного з книжками, після чого натиснула на перше відповідне посилання та перевірила, що відкрилася сторінка з достатнім текстовим вмістом.

Третій сценарій був складнішим, оскільки він включав відкриття категорії, перехід на сторінку товару та спробу додавання книжки до кошика. У цьому тесті я перевірила, що користувач може перейти до товару, побачити його сторінку та натиснути кнопку купівлі або додавання до кошика.

Після написання тестів я запустила їх у терміналі командою:


```

• oksanka@debian:~/Documents/automated_testing/e2e-testing$ npm test

> e2e-testing@1.0.0 test
> jest

PASS ./e2e.test.js (54.239 s)
  Vivat E2E user scenarios
    ✓ Scenario 1: user can search for books on the website (39072 ms)
    ✓ Scenario 2: user can open books category and navigate to a product page (6271 ms)
    ✓ Scenario 3: user can open product page and try to add a book to cart (8619 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        54.276 s
Ran all test suites.
• oksanka@debian:~/Documents/automated_testing/e2e-testing$

```

Після успішного запуску тестів я підготувала проєкт до завантаження на GitHub. У мене раніше уже був ініціалізований Git-репозиторій для виконання цих робіт.

Тому першим ділом я перевірила стан файлів:

```

oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Невідстежувані файли:
  (скористайтесь "git add <файл>...", щоб додати до майбутнього коміту)
    e2e-testing/

нічого не додано до коміту, але є невідстежувані файли (скористайтесь "git add" для відстежування)
oksanka@debian:~/Documents/automated_testing$

```

Далі я додала всі файли до індексації Git:

```

oksanka@debian:~/Documents/automated_testing$ git add .
oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Зміни, додані до майбутнього коміту:
  (використовуйте "git restore --staged <файл>...", щоб видалити з індексу)
    новий файл:      e2e-testing/.gitignore
    новий файл:      e2e-testing/e2e.test.js
    новий файл:      e2e-testing/package-lock.json
    новий файл:      e2e-testing/package.json

oksanka@debian:~/Documents/automated_testing$

```

Після цього я створила коміт:

```

oksanka@debian:~/Documents/automated_testing$ git commit -m "Add E2E user scenario tests for lab 6"
[main c27a8f1] Add E2E user scenario tests for lab 6
4 files changed, 5417 insertions(+)
 create mode 100644 e2e-testing/.gitignore
 create mode 100644 e2e-testing/e2e.test.js
 create mode 100644 e2e-testing/package-lock.json
 create mode 100644 e2e-testing/package.json
oksanka@debian:~/Documents/automated_testing$

```

Потім я завантажила проєкт на GitHub:

```
oksanka@debian:~/Documents/automated_testing$ git push -u origin main
Username for 'https://github.com': OksanaMykytyn
Password for 'https://OksanaMykytyn@github.com':
Перерахування об'єктів: 8, готово.
Підрахунок об'єктів: 100% (8/8), готово.
Дельта компресія з використанням до 4 потоків
Компресія об'єктів: 100% (6/6), готово.
Запис об'єктів: 100% (7/7), 44.35 KiB | 44.35 MiB/c, готово.
Всього 7 (дельта 1), повторно використано 6 (дельта 0), повторно використано пакунків 0
(з 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/OksanaMykytyn/automated_testing.git
   39d0a2d..c27a8f1  main -> main
Гілку "main" налаштовано на відстежування "origin/main".
oksanka@debian:~/Documents/automated_testing$
```

Посилання на GitHub:

https://github.com/OksanaMykytyn/automated_testing

Висновок: під час лабораторної роботи я навчилася створювати E2E-тести для перевірки користувацьких сценаріїв. Я обрала сайт Видавництва Віват, оскільки він має структуру інтернет-книгарні та дозволяє перевірити реальні дії користувача. Я реалізувала три сценарії: пошук книжки, перехід у категорію та відкриття товару, а також спробу додавання книжки до кошика. Під час написання тестів я використала Puppeteer для керування браузером і Jest для організації перевірок. Я створила допоміжні функції, щоб зменшити дублювання коду та зробити сценарії більш зрозумілими.